55. Capstone: Mini RAG / Embedding Search Service

55. Capstone: Mini RAG / Embedding Search Service

Phase: Phase 8 — Capstones
Estimated time: 4–6 days
Slug: `55-capstone-ai`

Learning objectives

- Implement chunk→embed→retrieve→answer orchestration
- Serve concurrent requests safely
- Add observability and eval harness
- Document failure modes

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Mission

Build a mini RAG service in Go:

- Ingest text documents (HTTP upload or CLI)
- Chunk + embed (embedder can be mocked/fake for tests)
- Query endpoint returns answer + citations
- Concurrent requests bounded by worker pool / semaphore
- Logs/metrics for latency and retrieval hit counts
- Offline eval over a tiny golden set

Milestones

1. Domain types + fake embedder/LLM
2. Chunk + store + cosine top-K
3. HTTP API + context timeouts
4. Concurrency limits + graceful shutdown
5. Eval harness + README architecture diagram

Rubric (excellent)

- Interfaces for embedder/LLM (swap real later)
- Race-clean
- Clear citation UX
- Eval numbers recorded in repo
- Thoughtful limits (max docs, max tokens)

Common mistakes (especially from Python instincts)

- Calling real paid APIs without mocks in unit tests

- No cancellation
- Unbounded document size

Exercises

Work in `exercises/55-capstone-ai/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/55-capstone-ai/` until you have an honest failing attempt.

Self-check

1. What did you mock and why?
2. How bound concurrency?
3. What eval metric did you pick?

Further reading

- https://go.dev/blog/context